

 FINE-TUNING

From Raw Data to Fine-Tuned LLM

The complete fine-tuning roadmap -
no gatekeeping, no hype.

SFT

LoRA

QLoRA

DPO

RLHF



AI Coach John

FOLLOW



PROITBRIDGE
Direct For Smarter Futures

 Repost

Fine-Tuning Is **Not Always** the Answer

Before investing time and compute into fine-tuning, verify that you actually need it. Many problems can be solved with simpler, cheaper techniques.

⚠ BEFORE YOU FINE-TUNE, TRY THESE FIRST

- > **Prompt engineering** → closes 80% of the gap, costs nothing
- > **RAG** → better for adding knowledge without retraining
- > **Larger base model** → may already solve the problem

DECISION FLOWCHART

Base model failing?

NO → Use as-is / RAG

↓ **YES**

Knowledge gap?

YES → Add RAG

↓

Style/format gap?

YES → Try prompt eng.

Prompt works? **YES** → Done

↓ **NO**

500+ quality examples?

NO → Collect data first

YES → **Fine-tune** ✓



AI Coach John

FOLLOW



 Repost

5 Situations Where Fine-Tuning Pays Off

1. Consistent output format

RAG+prompting breaks JSON schema 10% of the time → fine-tune for 99%+ compliance

2. Domain tone & voice

Legal, medical, brand-specific writing that prompts can't reliably enforce

3. Latency reduction

2,000-token system prompt → fine-tune it in → 90% shorter, faster inference

4. Cost at scale

100M calls/month → fine-tuned small model far cheaper than large model + long prompts

5. Security / compliance

Can't send data to cloud APIs → fine-tune a model that runs on-premise

ACCURACY VS EFFORT



AI Coach John

FOLLOW



Repost

Dataset Collection - Where Quality Data Lives

👑 TIER 1 - GOLD

- > Domain expert annotations
- > Human-reviewed production logs

👍 TIER 2 - GOOD

- > Production logs rated $\geq 4/5$
- > Curated: Alpaca, Dolly, FLAN

✂️ TIER 3 - WITH QA

- > Synthetic (GPT-4 / Claude)
- > Web-scraped + heavy cleaning

🔧 MINIMUM DATASET SIZE

Classification	500-1K
NER	500-2K
Instruction	1K-3K
Domain know.	2K-5K
Code gen.	5K-10K

```
def collect_training_examples(production_db,
    min_quality_score: float = 4.0,
    limit: int = 10000) → list[dict]:
    records = production_db.query("""
        SELECT user_message, assistant_response
        FROM interactions
        WHERE quality_score ≥ %(min_score)s
        AND human_reviewed = true
        ORDER BY quality_score DESC
        LIMIT %(limit)s
    """, {"min_score": min_quality_score, "limit": limit})
    return [{"instruction": r.user_message,
        "response": r.assistant_response} for r in records]
```



Dataset Cleaning - Garbage In, Garbage Out

WHAT TO REMOVE

- > Duplicate examples (inflate metrics, hurt generalization)
- > Low-quality responses (rushed, incomplete, factually wrong)
- > PII - emails, phone numbers, names (GDPR/HIPAA)
- > Formatting inconsistencies (same task, 3 different formats)
- > Toxic or biased examples

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")

def clean_dataset(examples: list[dict]) → list[dict]:
    # Remove semantic duplicates
    embeddings = model.encode(
        [e["instruction"] for e in examples])
    sim_matrix = cosine_similarity(embeddings)

    to_remove = set()
    for i in range(len(sim_matrix)):
        for j in range(i+1, len(sim_matrix)):
            if sim_matrix[i][j] > 0.95: # 95% similar = dup
                to_remove.add(j)
```

 **Most raw datasets have 20–40% duplicates. Always clean before training - it's the highest-ROI data step.**



Instruction Formatting - Teaching the Right Behavior

! WHY FORMAT MATTERS

If training format $\neq$ inference format, the model performs badly even with perfect data. Standardize format first.

STANDARD FORMATS

```
# Alpaca (simple, most tasks)
alpaca = {"instruction": "Classify sentiment.",
         "input": "Product broke in 2 days.",
         "output": "Negative"}

# ShareGPT (multi-turn)
sharegpt = {"conversations": [
    {"from": "human", "value": "Explain RAG."},
    {"from": "gpt", "value": "RAG retrieves docs ..."}
]}

# ChatML (modern, recommended)
chatml = [
    {"role": "system", "content": "You are helpful."},
    {"role": "user", "content": "What is fine-tuning?"},
    {"role": "assistant", "content": "Fine-tuning adapts ..."}
]
```

★ **Rule:** Pick one format. Apply it to 100% of your examples. Never mix formats in the same training run.



SFT - Supervised Fine-Tuning Foundation

WHAT SFT DOES

- > Trains on (instruction → response) pairs
- > Uses standard next-token prediction loss
- > Teaches domain format, tone, terminology
- > **SFT alone solves 80% of use cases**

WHEN TO STOP AT SFT

- > Format compliance tasks
- > Domain adaptation
- > Instruction following style

</> SFT WITH TRL

```
from trl import SFTTrainer
from transformers import (
    AutoModelForCausalLM,
    TrainingArguments)

model = AutoModelForCausalLM\
    .from_pretrained("Llama-3.1-8B")

args = TrainingArguments(
    output_dir="./sft_output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    warmup_ratio=0.03,
)

trainer = SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    max_seq_length=2048,
)

trainer.train()
```



Epochs: 1 = underfit, 3 = sweet spot, 10+ = catastrophic forgetting. Start at 3.



AI Coach John

FOLLOW



PROITBRIDGE
Simpler For Smarter Systems

 Repost

LoRA - Fine-Tune Without Fine-Tuning Everything

HOW LORA WORKS

Add small trainable matrices ($A \times B$) alongside frozen model weights. Only train the adapters - just **0.07% of parameters**. Full model stays frozen.

```
FROZEN (W)
Input → + → Output
      ↗ Low-rank adapter ↘
      A (d×r) B (r×d)
      TRAINABLE TRAINABLE
      r = 8-64 (rank)
```

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,          # Rank - lower = fewer params
    lora_alpha=32,  # Scaling factor
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none"
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable: 4.7M / 6.7B total (0.07%)
```

 **LoRA: 7B model from 56GB → 8GB VRAM. 10× cheaper, nearly same accuracy.**



QLoRA - Fine-Tune 70B on a Single GPU

WHAT QLORA DOES

Quantize the base model to **4-bit**, then add LoRA adapters on top. Train only the adapters - base stays in 4-bit frozen memory.

GPU MEMORY COMPARISON

Model	FP16	LoRA	QLoRA✓
7B	14GB	12GB	5GB <RTX4090
13B	26GB	22GB	8GB <RTX4090
70B	140GB	120GB	35GB <A100

```
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",      # NormalFloat4 (best)
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True, # Nested quantization
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/llama-3.1-70B",
    quantization_config=bnb_config,
    device_map="auto"
)
model = get_peft_model(model, lora_config)
```



DPO - Teach Preferences **Without a Reward Model**

📌 WHAT DPO DOES

Train on **chosen vs rejected** response pairs. Model learns to prefer human-preferred outputs - without PPO complexity.

🕒 WHEN TO USE

- After SFT - alignment step
- Better tone and safety
- Format preference tuning

</> DPO CODE

```
from trl import DPOTrainer

# Data format
pref_data = [{
    "prompt": "Explain QC",
    "chosen": "Quantum uses qubits...",
    "rejected": "Super complex..."
}]

dpo_trainer = DPOTrainer(
    model=sft_model,
    ref_model=ref_model, # frozen
    args=training_args,
    beta=0.1, # KL penalty
    train_dataset=pref_data,
    tokenizer=tokenizer,
)
dpo_trainer.train()
```



DPO vs RLHF: DPO skips the reward model entirely. Same alignment quality, 3× simpler to implement.



AI Coach John

FOLLOW



🔄 Repost

STEP 8

RLHF vs RLAIF - Human vs AI Feedback

Reinforcement learning from feedback is the final polish that aligns a model with human values, but collecting human feedback is notoriously slow and expensive.

RLHF (HUMAN)

Humans rate responses → reward model → PPO optimizes policy. Used by OpenAI & Anthropic.

Cost: Very High

Quality: Gold standard

Scale: Very Hard

RLAIF (AI JUDGE)

Replace human raters with a stronger LLM judge. Surprisingly competitive at scale.

Cost: Medium

Quality: Within 10% of RLHF

Scale: Easy

PIPELINE COMPARISON

RLHF: SFT → Samples → HUMANS rate → Reward Model → PPO

RLAIF: SFT → Samples → LLM-JUDGE rates → Reward Model → PPO



Use RLHF for safety-critical, budget >\$500K. Use RLAIF for most product teams.



AI Coach John

FOLLOW



↻ Repost

Evaluation - How Do You Know It's Better?

⚠ THE TRAP

Loss going down $\neq$ model is better at your task. Always evaluate on **held-out test data** with task-specific metrics.

BASELINE VS FINE-TUNED (500-EXAMPLE TEST SET)



```
from anthropic import Anthropic
client = Anthropic()

def evaluate_with_judge(instruction, response, criteria):
    result = client.messages.create(
        model="claude-opus-4-8",
        max_tokens=500,
        system=f"Expert evaluator. Rate on: {criteria}",
        messages=[{"role": "user", "content":
            f"Instruction: {instruction}\n"
            f"Response: {response}\n"
            "Rate 1-5 with reason."}]
    )
    return parse_scores(result.content[0].text)
```



Deploying Your **Fine-Tuned Model**

1. Merge LoRA adapters

→ Single model file, standard deployment, no adapter overhead

2. Serve with adapters (multi-tenant)

→ Base model shared, swap adapters per client - cost-efficient at scale

3. Quantize for production (4-bit)

→ 3× faster, 75% memory reduction, minimal accuracy loss

4. Cloud API (managed)

→ Together AI, Replicate, Fireworks AI - no infra to manage

```
from peft import PeftModel

# Merge adapters → single model file
base = AutoModelForCausalLM.from_pretrained("base_path")
peft_model = PeftModel.from_pretrained(base, "adapter_path")
merged = peft_model.merge_and_unload()
merged.save_pretrained("./my_finetuned_model")
```



Serving frameworks: vLLM (fastest) · TGI (HuggingFace) · Ollama (local) · Ray Serve (distributed)



Mistakes That Waste Months and Money

Fine-tuning involves many moving parts. A single misconfiguration in data formatting, hyperparameters, or evaluation can ruin the entire training run.

- 1 **Skipping the baseline** - always measure prompt eng. first. Fine-tuning solves what prompts can't.
- 2 **Wrong epoch count** - 1 epoch underfits. 10+ = catastrophic forgetting. Start at 3.
- 3 **Learning rate too high** - destroys general capabilities. Use $1e-4$ to $2e-4$ for LoRA.
- 4 **Mixed data formats** - Alpaca + ChatML in same run = confused model. Standardize everything.
- 5 **No eval set** - always hold out 10-20%. Training blind is not fine-tuning, it's gambling.
- 6 **Eval on training data** - loss looks great, real perf terrible. Use truly held-out test set.
- 7 **Catastrophic forgetting** - add 10-20% general instruction data to preserve base capabilities.



Fine-Tuning Roadmap Cheatsheet

STEP	WHAT	KEY TOOL
1. Decide	Is fine-tuning needed?	Decision framework
2. Collect	500+ quality examples min	HuggingFace Datasets
3. Clean	Dedup, PII, quality filter	SentenceTransformers
4. Format	Consistent chat template	Custom scripts
5. SFT	Teach task behavior	TRL SFTTrainer
6. LoRA	Reduce GPU memory 10×	PEFT library
7. QLoRA	Fine-tune 70B on 1 GPU	BitsAndBytes
8. DPO	Align preferences	TRL DPOTrainer
9. Evaluate	LLM-judge + human eval	Claude Opus as judge
10. Deploy	Merge adapters, serve	vLLM, Together AI

💡 The 80% Rule

SFT + LoRA solves 80% of use cases. Add DPO for alignment. Only add RLHF for safety-critical needs with the budget to match.

